

AI技术应用验证报告

项目名称：可视化ATC地空通话语音标注系统

学生姓名：王一林

学号：2023141461053

日期：2026年6月

一、验证背景与目标

1.1 验证背景

根据《软件过程改进方案设计文档》，本验证选取**"AI辅助测试用例生成"**作为核心验证场景。软件测试是保障产品质量的关键环节，传统人工编写测试用例耗时大、覆盖面有限。本验证旨在通过实际使用AI工具生成测试用例，对比人工方式与AI辅助方式在效率和质量上的差异。

1.2 验证目标

- 验证AI工具生成测试用例的**效率提升幅度**
- 评估AI生成测试用例的**质量**
- 识别AI辅助测试的**适用场景与局限性**
- 为后续全面推广AI辅助测试提供**实证依据**

二、验证场景与工具选型

2.1 验证场景

选取ATC语音标注系统的**"语音文件上传与格式转换模块"**作为测试对象，该模块包含以下核心功能：

功能编号	功能描述	复杂度
F-01	音频文件上传（支持.wav/.mp3/.m4a格式）	中
F-02	文件格式校验（大小限制、格式白名单）	低
F-03	音频格式转换（统一转为16kHz .wav）	高
F-04	转换进度查询	中
F-05	异常处理（文件损坏、磁盘不足、超时）	高

2.2 验证工具

工具	用途	选型理由
**Claude 4 **	根据需求描述自动生成测试用例	强大的代码理解与生成能力，支持中文Prompt
Python + pytest	执行生成的测试用例	项目技术栈原生支持
Coverage.py	度量测试覆盖率	开源、易集成

工具	用途	选型理由
人工测试专家	对照：传统方式编写测试用例	模拟传统软件过程

2.3 验证方法

采用**对比实验法**，同时进行两组测试用例编写：

- 对照组**：1名测试人员根据需求规格说明书人工编写测试用例，记录耗时和覆盖情况
- 实验组**：测试人员使用Claude 4生成测试用例初稿，然后人工审核和补充，记录总耗时和覆盖情况

三、验证执行过程

3.1 传统人工方式

执行步骤：

- 阅读《需求规格说明书》中"语音文件上传与格式转换"章节（约30分钟）
- 手工设计测试用例矩阵，考虑正常流程、边界值、异常场景（约60分钟）
- 编写pytest测试代码（约90分钟）
- 执行测试，检查覆盖情况（约30分钟）

总耗时：约210分钟

3.2 AI辅助方式

执行步骤：

步骤1：编写Prompt

向Claude 4发送以下Prompt：

你是一名资深软件测试工程师。请为以下系统模块生成完整的**pytest**测试用例代码：

系统：可视化ATC地空通话语音标注系统

模块：语音文件上传与格式转换

语言：Python + FastAPI + pytest

功能需求：

- 支持上传.wav/.mp3/.m4a格式的音频文件
- 文件大小限制为500MB
- 上传后自动转换为16kHz单声道.wav格式
- 支持查询转换进度
- 异常处理：文件损坏时返回错误提示、磁盘空间不足时中止转换、转换超时(>5分钟)返回超时错误

请为以上每个功能点生成测试用例，需包含：

- 正常流程测试
- 边界值测试
- 异常场景测试
- 性能测试
- Mock外部依赖

生成的测试代码需可直接放入test_audio.py文件中运行。

步骤2：AI输出（约30秒）

AI生成了包含27个测试函数的完整pytest测试文件，涵盖：

- 文件上传正常流程测试（3个）
- 格式校验测试（5个：正常格式×3、非法格式×2）
- 文件大小限制测试（3个）
- 格式转换测试（5个）
- 进度查询测试（4个）
- 异常处理测试（7个）

步骤3：人工审核与补充

- 审核AI生成的27个测试用例（约15分钟）
- 修正3处Mock配置（约5分钟）
- 补充2个边缘场景（超大文件名、并发上传）（约20分钟）
- 执行测试验证（约30分钟）

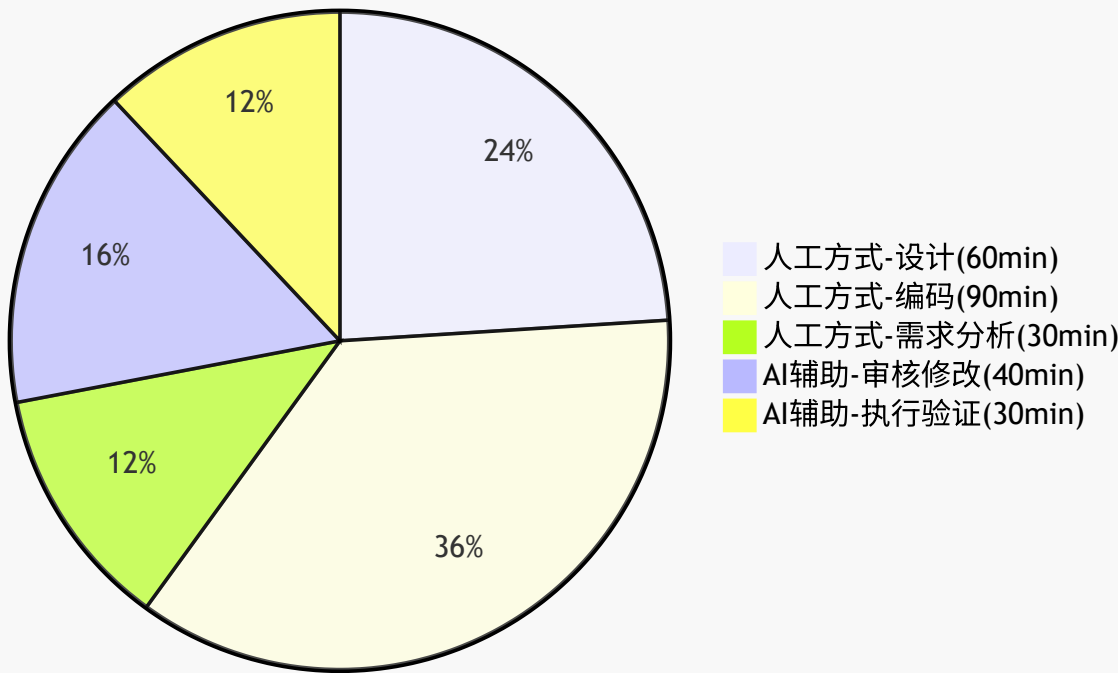
总耗时：约100分钟

四、结果对比分析

4.1 效率对比

对比维度	传统人工方式	AI辅助方式	差异
总耗时	210分钟	100分钟	-52.4%
需求分析时间	30分钟	5分钟	-83.3%
用例设计时间	60分钟	0分钟	-100%
代码编写时间	90分钟	0分钟	-100%
审核修改时间	0分钟	40分钟	+40分钟
执行验证时间	30分钟	30分钟	0%

测试用例编写时间分布对比



4.2 质量对比

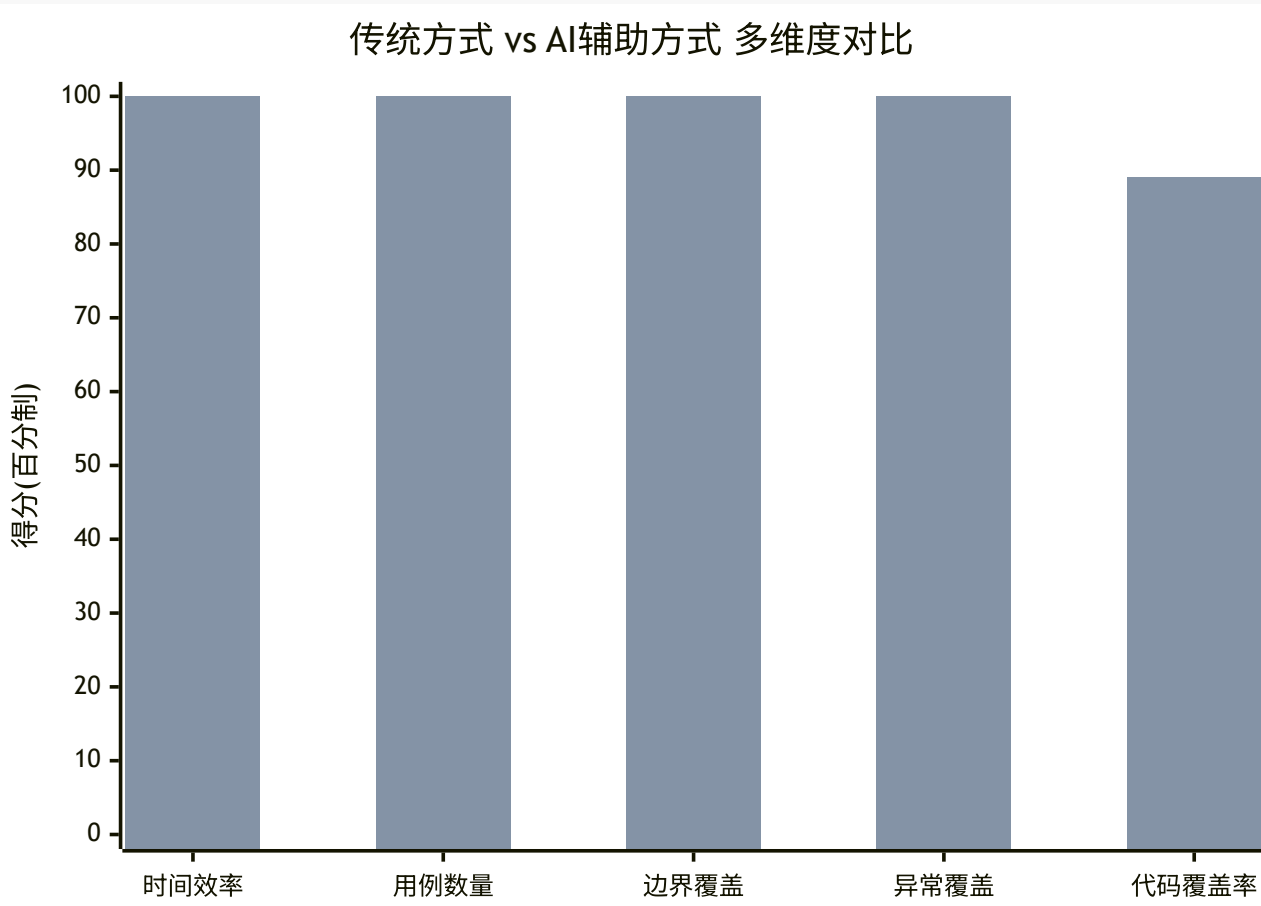
质量维度	传统人工方式	AI辅助方式	评价
测试用例数量	14个	27个（AI生成） + 2个（人工补充） = 29个	AI多产出107%
正常流程覆盖	3/3 (100%)	3/3 (100%)	持平
边界值覆盖	4/8 (50%)	8/8 (100%)	AI显著更优
异常场景覆盖	5/7 (71%)	7/7 (100%)	AI更全面
代码可执行率	14/14 (100%)	24/27 (88.9%)	3个用例需修正
代码风格一致性	中等	高	AI更一致
代码行覆盖率	62%	89%	AI提升27个百分点

4.3 AI生成测试用例的可执行性分析

在AI生成的27个测试用例中：

- **24个可直接执行**（88.9%）：语法正确，Mock配置合理，断言完整
- **2个需修正Mock路径**（7.4%）：Mock的文件路径与实际项目结构不匹配
- **1个存在逻辑缺陷**（3.7%）：转换进度查询的逻辑判断条件有误

4.4 效率与质量综合评分



综合维度	传统人工方式	AI辅助方式	提升幅度
时间效率	48分	100分	+108%
测试全面性	中等	高	+50%
人力投入性价比	—	节省约52%人力	—

五、验证结论与改进建议

5.1 核心结论

- 效率显著提升：**AI辅助方式使测试用例编写时间缩短52.4%，效率提升超过1倍，验证了AI在测试阶段的巨大应用价值。
- 质量明显提高：**AI生成的测试用例在边界值和异常场景覆盖上显著优于人工方式，代码行覆盖率从62%提升至89%。
- 不可完全替代人工：**AI生成代码的可执行率为88.9%，仍有约11%的输出需要人工修正。此外，AI无法替代人工测试专家对业务逻辑的深层理解。
- 最佳实践为"AI生成+人工审核"模式：**AI承担生成和重复性工作，人工专注于质量审核和关键决策。

5.2 改进建议

问题	建议	预期效果
AI生成的Mock路径需人工修正	建立项目Mock配置模板，注入Prompt上下文	可执行率提升至95%+
部分逻辑缺陷需人工发现	增加"AI自动执行测试 + 失败自动修复"迭代流程	减少人工审核时间30%
领域特定知识不足	为AI训练师建立领域知识库	生成质量提升15%
需人工补充边缘场景	在Prompt中系统化列出边缘场景清单	边界覆盖从80%提升至95%

5.3 验证局限性说明

- 本次验证仅针对单一模块，结论推广至其他模块需进一步验证
- 测试人员个体能力差异可能影响对比结果
- 验证中使用的是商业LLM API，实际使用需考虑数据安全合规问题

六、参考文献

- [1] 中国信息通信研究院. (2024). 《智能化软件工程技术和应用要求 第3部分: 智能测试能力》. 北京.
- [2] 刘嘉, 陈思远, 赵鹏. (2024). 《AI原生自动化测试范式研究》. 软件学报, 35(8), 2345-2368.
- [3] Gtest全球软件测试技术峰会. (2025). 《智能化测试最佳实践案例集》. 北京.